

```

1  unit Form_Main;
2  {
3  Created 24/10/2023 By Roger Williams
4
5  Functions:
6
7  - Creates main database using SQLite, getting table definitions from
  ..\Database\schemas.txt
8  - TEMP: creates reports in Excel ultimately will use custom written report system
9  - Export reports to PDF
10
11 Files copied into Database\Imports
12 Folder structure:
13
14                                     Database
15                                     |      (schemas.txt and
16                                     |      rogoClock_database.db)
17                                     |
18                                     |-----|
19                                     Imports
20                                     MainDB_Backups
21
22                                     Contains:
23                                     Contains:
24
25 attendancelog.dat_<date>_<time>   ClockMachineDB_<date>_<time>
26 rogoClock_database_<date>_<time>.db
27                                     (1_attlog.dat)                (ZKDB.db)
28                                     (rogoClock_database.db)
29
30
31 NOTE: in the ZKDB.db->USER_INFO table ValidCount is spelt VAILDCount!
32
33 }
34 interface
35
36 uses
37   Winapi.Windows, Winapi.Messages, System.SysUtils, System.Variants, System.Classes, Vcl.
  Graphics,
38   Vcl.Controls, Vcl.Forms, Vcl.Dialogs, FireDAC.Stan.ExprFuncs,
39   FireDAC.Phys.SQLiteWrapper.Stat, FireDAC.Phys.SQLiteDef, FireDAC.Stan.Intf,
40   FireDAC.Stan.Option, FireDAC.Stan.Error, FireDAC.UI.Intf, FireDAC.Phys.Intf,
41   FireDAC.Stan.Def, FireDAC.Stan.Pool, FireDAC.Stan.Async, FireDAC.Phys,
42   FireDAC.Phys.SQLite, FireDAC.VCLUI.Wait, FireDAC.Stan.Param, FireDAC.DatS,
43   FireDAC.DApt.Intf, FireDAC.DApt, Vcl.Menus, System.ImageList, Vcl.ImgList,
44   FireDAC.Comp.Client, Data.DB, FireDAC.Comp.DataSet, DateUtils, System.IOUtils,
45   Excel2010;
46
47 type
48   TFRMMain = class(TForm)
49     DataSource1: TDataSource;
50     FDPhysSQLiteDriverLink1: TFDPhysSQLiteDriverLink;
51     FDConnection1: TFDConnection;
52     FDQuery1: TFDQuery;
53     FDSchemaAdapter1: TFDSchemaAdapter;
54     IMLMenu: TImageList;
55     MMUMain: TMainMenu;
56     MNUFile: TMenuItem;
57     MNUExit: TMenuItem;
58     MNUTools: TMenuItem;
59     MNUCreateMainDatabase: TMenuItem;
60     MNUImportData: TMenuItem;
61     procedure MNUCreateMainDatabaseClick(Sender: TObject);
62     procedure MNUExitClick(Sender: TObject);

```

```

63     procedure MNUImportDataClick(Sender: TObject);
64 private
65     { Private declarations }
66     function CreateMainDatabase:boolean;
67
68     const CNST_STR_DATABASEPATH = 'C:\ROGoClockTimeandAttendance\Database\';
69     const CNST_STR_DATABASEBACKUPPATH =
70         'C:\ROGoClockTimeandAttendance\Database\MainDB_Backups\';
71     const CNST_STR_IMPORTSPATH = 'C:\ROGoClockTimeandAttendance\Database\Imports\';
72     const CNST_STR_APPLICATIONPATH = 'C:\ROGoClockTimeandAttendance\';
73     const CNST_STR_DATABASENAME = 'rogoclock_database.db';
74
75 public
76     { Public declarations }
77 end;
78
79 var
80     FRMMain: TFRMMain;
81
82 implementation
83
84     {$R *.dfm}
85
86     function TFRMMain.CreateMainDatabase:boolean;
87     {
88     Created 24/10/2023 By Roger Williams
89
90     Creates main database using SQLite, getting table definitions from
91     ..\Database\schemas.txt
92
93     Schema file excerpt:
94
95     CREATE TABLE USER_INFO_DEPT
96     (
97         [ID] integer PRIMARY KEY AUTOINCREMENT,
98         [UID_Dept] varchar(48),
99         [UID_UserID] integer
100     )
101
102
103 }
104 var
105     strTemp,
106     strSQL,
107     strTable,
108     strField : string;
109     intNum    : integer;
110     strldata  : TStringList;
111     tblTemp   : TFDTable;
112     intErr    : integer;
113
114 begin
115     intErr:=0;
116     //make backup of current database
117     if FileExists(CNST_STR_DATABASEPATH+'rogoclock_database.db') then
118     begin
119         if DayOf(Date) <10 then
120             strTemp:='0'+DayOf(Date).ToString
121         else
122             strTemp:=DayOf(Date).ToString;
123
124         if MonthOf(Date) <10 then
125             strTemp:=strTemp+'0'+MonthOf(Date).ToString
126         else
127             strTemp:=strTemp+MonthOf(Date).ToString;
128
129         strTemp:=strTemp+YearOf(Now).ToString + '_';

```

```

130 //add time to end of file
131 if HourOf(Now) <10 then
132     strTemp:=strTemp+'0'+HourOf(Now).ToString
133 else
134     strTemp:=strTemp+HourOf(Now).ToString;
135
136 if MinuteOf(Now) <10 then
137     strTemp:=strTemp+'0'+MinuteOf(Now).ToString
138 else
139     strTemp:=strTemp+MinuteOf(Now).ToString;
140
141 if MinuteOf(Now) <10 then
142     strTemp:=strTemp+'0'+SecondOf(Now).ToString
143 else
144     strTemp:=strTemp+SecondOf(Now).ToString;
145
146 //make sure there is a main db backup directory
147 if not DirectoryExists(CNST_STR_DATABASEBACKUPPATH) then
148     TDirectory.CreateDirectory(CNST_STR_DATABASEBACKUPPATH);
149
150 try
151     CopyFile(PChar(CNST_STR_DATABASEPATH+CNST_STR_DATABASENAME),PChar(
        CNST_STR_DATABASEBACKUPPATH+TPath.GetFileNameWithoutExtension(CNST_STR_DATABASENAME) +
        strTemp + '.dbbak'),True);
152 except
153     on exception do
154         intErr:=1;
155 end;
156
157 try
158     //delete current database
159     DeleteFile(CNST_STR_DATABASEPATH+CNST_STR_DATABASENAME);
160 except
161     on exception do
162         intErr:=2;
163 end;
164
165 end;
166
167 //read schema file into memory
168 strlData:=TStringList.Create;
169
170 try
171     strlData.LoadFromFile(CNST_STR_DATABASEPATH+'schemas.txt');
172 except
173     on exception do
174         intErr:=3;
175 end;
176
177 if intErr =0 then
178 begin
179     //create database
180     self.FDConnection1.Params.Add('DriverID=SQLite');
181     self.FDConnection1.Params.Add('Database=' + CNST_STR_DATABASEPATH+
        CNST_STR_DATABASENAME);
182     self.FDConnection1.Open;
183
184 //Create Tables from string list
185 intNum:=0;
186
187 while intNum <> strlData.Count-1 do
188 begin
189     strTemp:=strlData[intNum];
190
191     if strTemp <> '' then
192         strSQL:=strSQL + strTemp
193     else
194         begin
195             strSQL:=strSQL + strTemp;

```

```

196         //create table using SQL query
197         self.FDConnection1.ExecSQL(strSQL);
198         strSQL:='';
199     end;
200
201     inc(intNum);
202 end;
203
204     self.FDConnection1.Close;
205     //free string list
206     strldata.Free;
207 end;
208
209 result := intErr=0;
210 end;
211
212 procedure TFRMMain.MNUCreateMainDatabaseClick(Sender: TObject);
213 begin
214     if application.MessageBox('This Will Delete Any Existing Database - Are You Sure?',
215     'Create Main Database',mb_YesNo) = mrYes then
216     begin
217         if CreateMainDatabase then
218             ShowMessage('Database Created!'+chr(10)+chr(13)+'Don't forget update the tables
219             using the import function');
220         end;
221     end;
222
223     procedure TFRMMain.MNUExitClick(Sender: TObject);
224     begin
225         self.Close;
226     end;
227
228     procedure TFRMMain.MNUImportDataClick(Sender: TObject);
229     {
230         Created 30/10/2023 By Roger Williams
231
232         reads the ZKDB.db file created by a TA-8015 from data mgt->backup to USB
233
234         - copies the ZKDB.db to the Database\Imports folder giving the ZKDB.db
235         todays date and time
236         - deletes matching user in the rodoclock_database.db
237         - creates users from ZKDB.db
238     }
239
240     var
241         FDConnROG,
242         FDConnTA          : TFDConnection;
243         FDQueryROG,
244         FDQueryTA         : TFDQuery;
245         FDMEMResultsTA,
246         FDMEMResultsROG  : TFDMemTable;
247         strTemp,
248         strTemp2,
249         strSQL,
250         strExt            : string;
251         intNum            : integer;
252         ODLGOpen          : TOpenDialog;
253         blnOk             : Boolean;
254
255     begin
256         blnOk:=True;
257         //first get location of the ZKDB.db file
258         ODLGOpen:=TOpenDialog.Create(self);
259         ODLGOpen.InitialDir:='C:\';
260         ODLGOpen.Title:='Select ZKDB.db File';
261         ODLGOpen.DefaultExt:='.db';
262         ODLGOpen.Filter:='TA-8015 Database Files|zkdb.db';

```

```

263
264 if ODLGOpen.Execute then
265 begin
266     strExt:=TPath.GetExtension(ODLGOpen.FileName);
267     strTemp:=TPath.GetFileNameWithoutExtension(ODLGOpen.FileName)+'_';
268     //add todays date and time to filename
269     if DayOf(Now) <10 then
270         strTemp:=strTemp+'0'+DayOf(Now).ToString
271     else
272         strTemp:=strTemp+DayOf(Now).ToString;
273
274     if MonthOf(Now) <10 then
275         strTemp:=strTemp+'0'+MonthOf(Now).ToString
276     else
277         strTemp:=strTemp+MonthOf(Now).ToString;
278
279     strTemp:=strTemp+YearOf(Now).ToString+'_';
280     //add time to end of file
281     if HourOf(Now) <10 then
282         strTemp:=strTemp+'0'+HourOf(Now).ToString
283     else
284         strTemp:=strTemp+HourOf(Now).ToString;
285
286     if MinuteOf(Now) <10 then
287         strTemp:=strTemp+'0'+MinuteOf(Now).ToString
288     else
289         strTemp:=strTemp+MinuteOf(Now).ToString;
290
291     if MinuteOf(Now) <10 then
292         strTemp:=strTemp+'0'+SecondOf(Now).ToString
293     else
294         strTemp:=strTemp+SecondOf(Now).ToString;
295
296     //put file extension back
297     strTemp:=strTemp+strExt;
298
299     //copy to imports folder
300     try
301         TFile.Copy(ODLGOpen.FileName,CNST_STR_IMPORTSPATH+strTemp,True);
302     except
303         on exception do
304             blnOk:=false;
305     end;
306
307     //only continue if file copied
308     if blnOk then
309     begin
310         FDConnTA:=TFDConnection.Create(self);
311         FDConnROG:=TFDConnection.Create(self);
312         //open database to import
313         FDConnTA.Params.Add('DriverID=SQLite');
314         FDConnTA.Params.Add('Database=' + CNST_STR_IMPORTSPATH+strTemp);
315         FDConnTA.Open;
316         //open main database
317         FDConnROG.Params.Add('DriverID=SQLite');
318         FDConnROG.Params.Add('Database=' + CNST_STR_DATABASEPATH+CNST_STR_DATABASENAME);
319         FDConnROG.Open;
320
321         //delete all users from main db that are in the import db
322         //NOTE: keep DEPT and USER_INFO_MORE records!
323
324         //query db to import
325         FDQueryTA:=TFDQuery.Create(self);
326         FDQueryTA.Connection:=FDConnTA;
327         FDQueryTA.SQL.Add('SELECT * FROM USER_Info ORDER BY User_PIN');
328         //run query put results into temp table in memory
329         FDQueryTA.Open;
330         FDMEMResultsTA:=TFDMemTable.Create(nil);
331         FDMEMResultsTA.FieldDefs.Assign(FDQueryTA.FieldDefs);

```

```

332 FDMEMResultsTA.CreateDataSet;
333 FDMEMResultsTA.CopyDataSet(FDQueryTA.Fields.DataSet);
334 FDMEMResultsTA.First;
335 //query main db
336 FDQueryROG:=TFDQuery.Create(self);
337 FDQueryROG.Connection:=FDConnROG;
338 FDQueryROG.SQL.Add('SELECT * FROM USER_Info ORDER BY User_PIN');
339 //run query put results into temp table in memory
340 FDQueryROG.Open;
341 FDMEMResultsROG:=TFDMemTable.Create(nil);
342 FDMEMResultsROG.FieldDefs.Assign(FDQueryROG.FieldDefs);
343 FDMEMResultsROG.CreateDataSet;
344 FDMEMResultsROG.CopyDataSet(FDQueryROG.Fields.DataSet);
345 FDMEMResultsROG.First;
346
347 //iterate through table looking for user to import that MATCH existing user
348 while not FDMEMResultsTA.Eof do
349 begin
350 //if found delete from User_Info in main database
351 if FDMEMResultsROG.Locate('User_PIN',FDMEMResultsTA.Fields.FindField('User_PIN').
Value) then
352     FDConnROG.ExecSQL('DELETE FROM User_Info WHERE User_PIN ='+FDMEMResultsTA.Fields
        .FindField('User_PIN').AsString+'');
353
354 //add User_Info data from import db into main db
355 //add here opposed to above because need to all existing AND new users to main db
356
357     strSQL:='INSERT INTO User_Info ' +
358         '(User_PIN, Privilege, Name, Password, Face_Group_ID,
359         Acc_Group_ID, Dept_ID, Is_Group_TZ,' +
360         'Verify_Type, Main_Card, Vice_Card, CREATE_ID, MODIFY_TIME,
361         SEND_FLAG, Expires, StartDateTime,' +
362         'EndDateTime, VaildCount, Timezone1, Timezone2, Timezone3) ' +
363         'VALUES ('' ' +
364         FDMEMResultsTA.Fields.FindField('User_PIN').AsString+'''+','+
365         FDMEMResultsTA.Fields.FindField('Privilege').AsString+'','''+
366         FDMEMResultsTA.Fields.FindField('Name').AsString+'','''+
367         FDMEMResultsTA.Fields.FindField('Password').AsString+'','+
368         FDMEMResultsTA.Fields.FindField('Face_Group_ID').AsString+','+
369         FDMEMResultsTA.Fields.FindField('Acc_Group_ID').AsString+','+
370         FDMEMResultsTA.Fields.FindField('Dept_ID').AsString+','+
371         FDMEMResultsTA.Fields.FindField('IS_Group_TZ').AsString+','+
372         FDMEMResultsTA.Fields.FindField('Verify_Type').AsString+'','''+
373         FDMEMResultsTA.Fields.FindField('Main_Card').AsString+'','''+
374         FDMEMResultsTA.Fields.FindField('Vice_Card').AsString+'','''+
375         FDMEMResultsTA.Fields.FindField('CREATE_ID').AsString+'','''+
376         FDMEMResultsTA.Fields.FindField('MODIFY_TIME').AsString+'','+
377         FDMEMResultsTA.Fields.FindField('SEND_FLAG').AsString+','+
378         FDMEMResultsTA.Fields.FindField('Expires').AsString+'','''+
379         FDMEMResultsTA.Fields.FindField('StartDatetime').AsString+'','''+
380         +
381         FDMEMResultsTA.Fields.FindField('EndDatetime').AsString+'','+
382         FDMEMResultsTA.Fields.FindField('VaildCount').AsString+','+
383         FDMEMResultsTA.Fields.FindField('Timezone1').AsString+','+
384         FDMEMResultsTA.Fields.FindField('Timezone2').AsString+','+
385         FDMEMResultsTA.Fields.FindField('Timezone3').AsString+')';
386
387     FDConnROG.ExecSQL(strSQL);
388     FDMEMResultsTA.Next;
389 end;
390
391 FDConnRog.Commit;
392 FDQueryTA.Free;
393 FDQueryROG.Free;
394 FDConnTA.Close;
395 FDConnROG.Close;
396 FDConnTA.Free;
397 FDConnROG.Free;
398 Application.MessageBox('Import Completed','Success!',0);
399 end

```

```
386     else
387         MessageDlg('Import Failed',mtError,[mbOk],0);
388     end;
389
390     ODLGOpen.Free;
391     end;
392
393     end.
394
```